Selenium WebDriver User Guide

Acme Automation sample documentation | Version 2.4 | Updated: 15 October 2026

1. Purpose

Selenium WebDriver automates browser actions such as opening pages, locating elements, clicking buttons, entering text and reading page content. It is suitable for testing web applications through a real browser.

2. Prerequisites

Use Python 3.11 or 3.12, Selenium 4.20 or later, and a supported browser. Keep the browser reasonably up to date. Selenium Manager can help discover and manage browser drivers in supported setups.

3. Installation

Activate the project's virtual environment and run **python -m pip install selenium**. Verify the installation with **python -c "import selenium; print(selenium.__version__)"**.

4. Basic workflow

• Import webdriver from selenium.

• Create a browser instance, for example webdriver.Chrome().

• Navigate with driver.get(url).

• Locate elements with By.ID, By.NAME, By.CSS_SELECTOR or By.XPATH.

• Interact with elements and read their text or attributes.

• Close the browser with driver.quit() in a finally block.

5. Wait strategies

Prefer explicit waits using WebDriverWait and expected_conditions when an element must become visible, clickable or present. Avoid fixed time.sleep calls as the main synchronization strategy because page load times vary. The team's standard explicit-wait timeout is 10 seconds.

6. Locator guidance

Prefer stable unique IDs or accessible attributes. Use CSS selectors for readable and maintainable locators. Use XPath when relationships or text-based selection make it necessary. Avoid long absolute XPath expressions because small page changes can break them.

7. Browser cleanup

Always quit the WebDriver after the test, including when an exception occurs. Leaving browser processes running can consume memory and interfere with later test runs.

8. Example policy

For this sample project, use Chrome for browser automation. Firefox may be used for compatibility checks, but test results must record the browser name and version.